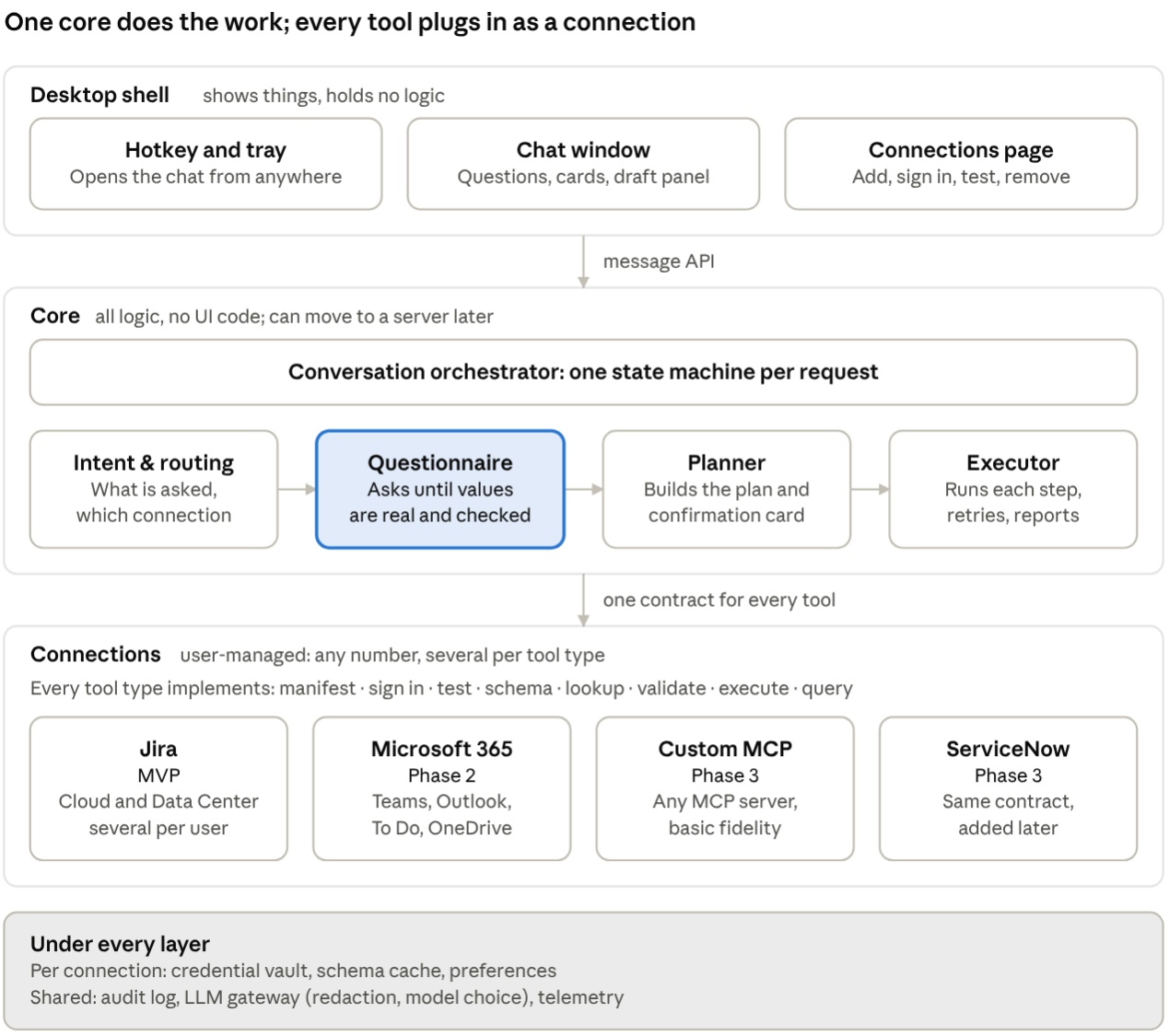
Desktop AI Task Assistant: Architecture & Tech Stack Options

Oct 12, 2026 · @Baranidharan Selvaraj

We are building a Windows assistant, opened with a global hotkey, that turns a chat request into checked, confirmed actions in the user's connected tools. The MVP connects to Jira, followed by Microsoft 365 and any MCP server. Before it acts, it asks questions until it has real, validated values for everything it needs. This doc sums up the design so far and compares the tech stacks we could build it with.

At a glance



architecture · shell, core, connections, shared services

The shell only displays things. The core holds all the logic and reaches tools only through the shared connector contract. The highlighted box, the questionnaire, is our main addition.

How a request flows

Every write goes through the same seven steps. Reads (status, search) skip steps 4 and 5 and answer straight away.

1. **Open.** The user presses the hotkey and types a request, e.g. "Create a bug for the login page crashing on Safari, high priority, frontend team".
2. **Understand and route.** The assistant extracts the operation (create), the item type (Bug), and each field along with how sure it is about it. It also decides which connection the request is for (here: Company Jira).
3. **Look things up.** It fetches what the tool already knows: projects, required fields for a Bug, allowed values, and people or components matching "frontend team". No questions are asked for anything it can look up.
4. **Ask.** It asks one question at a time, with real choices plus a free-text box, until it has enough information or the user presses **Stop asking**. After each answer it checks the value, updates the draft, and picks the next most useful question.
5. **Confirm.** A confirmation card shows exactly what will be done, where, and which values were assumed. The user confirms, edits or cancels.
6. **Do it.** The executor makes the change through the connection, acting as the user.
7. **Report.** The assistant replies with a link (e.g. WEB-1234) and writes an audit entry.

The user can type at any point to change details. A request that spans tools (create the bug, then post it on Teams) becomes one plan with several steps, confirmed once.

The questionnaire

The questionnaire is our main differentiator. The assistant asks one question at a time, with no limit, until every value it needs is real and checked, or until the user stops it.

Where questions come from, in the order they're asked:

#	Source	Example	Blocking?
1	Which tool	"Should this go to Company Jira or Client Jira?"	Yes
2	Unclear intent	"Create a new ticket or update WEB-1200?"	Yes

#	Source	Example	Blocking?
3	Missing required field	"Which version is affected?"	Yes
4	Ambiguous name	"Arjun Kumar (Frontend) or Arjun Rao (QA)?"	Yes
5	Invalid value	"'Urgent' isn't a priority here. Pick Highest, High or Medium."	Yes
6	Low confidence	"I took the priority as High. Correct?"	Yes
7	Quality	Bug: steps to reproduce, expected vs actual result	No, skippable

How it asks:

- **Choices plus free text on every question.** Choices are always real values from the tool (allowed values, people, projects). A typed answer is matched to a real value. If it matches several or none, the assistant asks a follow-up.
- **Never asks what it can look up.** Project lists, people search and allowed priorities are fetched, not asked.
- **Dependency order.** Tool, then project, then issue type, then that type's fields, because each answer changes what's required next.
- **Shows its working.** A live draft panel next to the chat shows every field filled so far, editable in place.
- **Remembers preferences.** Usual project, components, assignees and Teams channels are offered pre-selected, never applied silently.
- **Two profiles.** *Guided* asks more questions, including quality questions. *Expert* asks only blocking ones.

When the user presses Stop asking:

- Gaps are filled only from real sources, in this order: the tool's default for the field, then the user's saved preferences, then left empty (unassigned, no label).
- Each filled value is labelled **assumed** on the confirmation card.
- If a required field still has no real value, the card offers three options: answer just that one question, save as a draft, or open the tool's own form with everything prefilled. The assistant never invents a value to get past the tool's validation.

Connectors and connections

Tools are connections the user sets up and manages. The core never assumes Jira or any single tool.

- **Connector type:** code we write that knows one product's API, e.g. Jira or Microsoft 365. It ships in the app's catalogue.
- **Connection:** one account on one instance of a type, set up by the user, e.g. "Company Jira" and "Client Jira". A user can have as many as they like.

Every connector type implements the same contract:

Member	Purpose	Jira	Microsoft 365
manifest	Name, auth methods, item types, ID patterns, keywords	Issue; IDs like WEB-123; bug, story, sprint	Email, Teams message, meeting, To Do task, file, person
authenticate	Sign in as the user	API token, OAuth or PAT	Microsoft sign-in
testConnection	Prove it works and show who it acts as	"Connected as Priya"	"Connected as priya@company.com"
getSchema	Required fields and allowed values	createmeta, editmeta, transitions	Mostly fixed (email: to, cc, subject, body)
lookup	Turn names into real IDs	Users, projects, components, sprints, epics	People directory, teams and channels, files
validate	Check a draft before planning	Allowed values, formats	Recipients resolved, channel exists
execute	Make a change	Create, update, transition, comment	Send email, post message, create task or meeting
query	Answer reads	Natural language to JQL	Search mail, files, calendar
openInTool (optional)	Open the tool's own form, prefilled	Jira create screen	Outlook compose window

Adding a connection (Settings, or by asking in chat "connect my Microsoft 365"):

1. Pick a type from the catalogue, or Custom (MCP).
2. Enter its details, e.g. the site URL. Jira Cloud vs Data Center is detected automatically.

3. Sign in. OAuth opens the system browser, and tokens go straight into the Windows credential store.
4. Test: "Connected as …" or a plain-language error.
5. The assistant caches the tool's projects, fields, teams and channels.
6. Name the connection and set defaults, optionally making it read-only.

Routing: which connection is a request for? The first check that gives a confident answer wins:

1. The connection or product is named, or picked with an @ mention.
2. An ID in the message, e.g. WEB-123, belongs to one connection.
3. Only one connection can do this kind of task.
4. Keywords match, e.g. "sprint" for Jira, or "email" or "Teams" for Microsoft 365.
5. Otherwise the first question is "Which tool?", with the usual choice pre-selected.

Jira + Microsoft 365 workflows. Microsoft 365 is the people and collaboration layer next to Jira. One request can cover the whole workflow. "Log a bug for the Safari crash, attach the crash log from my OneDrive, and tell the frontend team on Teams" becomes three steps, confirmed once:

1. Find the file in OneDrive, asking which one if two match.
2. Create the Jira bug with the file attached.
3. Post the message on Teams, with its full text shown on the card for approval.

The company directory also improves people questions, showing job title and department. Each person's work email links them to their account in every tool.

Custom MCP connections cover tools we haven't built. A user can attach any MCP server, remote or local:

- Each server tool's input schema becomes the fields the questions are built from.
- Tools not marked read-only are treated as writes, so they always need confirmation.
- These connections are labelled "basic" because there are no lookups to offer real choices.
- A local server runs code on the user's machine, so it needs explicit consent.

Safety and the basics to handle

The rule behind all of these: the LLM proposes, the tool's own data decides. Nothing reaches a tool unless it came from the user, a lookup, or the tool's schema.

Area	What we do
Acting as the user	Each connection uses the user's own sign-in and permissions, never an admin account.
Credentials	Stored in the Windows credential store, per connection. Never logged, never sent to the LLM.
Confirmation	Every write is confirmed on a card. Deletes and bulk edits get an extra warning or are blocked in the MVP. Any connection can be set to read-only.
Messages to people	Emails, Teams messages and invites can't be unsent, so the card shows the exact recipients and full text. Shared files use organisation- or person-scoped links, never "anyone with the link".
Data sent to the LLM	Only what's needed, with redaction rules. Rules can differ per connection (e.g. a client's Jira under NDA). The provider is still to be chosen.
Audit	A local log of every write: the connection, the request, the approved plan, the API calls made, and the result.
Connection health	Each connection shows connected, needs re-auth or unreachable. Drafts are kept until a broken connection is fixed.
Errors	Per-connection rate limiting with back-off. In multi-step plans, the assistant reports what succeeded and offers to retry the rest.
Tool changes	Each connection's cached schema expires and is refreshed whenever a validation fails.
Speed and cost	A small model for routing and classification, a larger one only for planning and drafting messages.
Rollout	Signed installer, auto-update, Intune/SCCM support. IT usually has to approve the app once in Microsoft Entra ID before users can connect Microsoft 365.

Decisions so far

Eight design decisions are settled. The only open one is the LLM provider, which is part of the tech stack choice below.

Decision	Choice
Where the logic runs	On the user's machine, as a core module with no UI code, so it can move to a server later
Tools	User-managed connections: any number, several per type
Tools we haven't built	Our own catalogue, plus any custom MCP server
MVP tools	Jira only, with the full connections system
Microsoft	Microsoft 365 (Teams, Outlook, To Do, OneDrive, directory) as the collaboration layer next to Jira
Users	Both power users and requesters, via Guided and Expert profiles
Jira versions	Cloud and Data Center, detected per connection
Settings	Per user, stored locally, in a format central IT config could override later

Two defaults were chosen, and either can be changed:

- If the user stops with a required field missing, they can answer it, save a draft, or open the tool prefilled.
- Microsoft 365 comes before ServiceNow in the roadmap.

Still open: the LLM provider, and the company's rules on what data may leave the machine.

Phases

The MVP proves the connections system and the questionnaire on Jira alone. Each later phase adds reach.

1. **MVP:** hotkey, tray icon and chat. The full connections system (add, sign in, test, re-auth, remove, several connections, routing). The Jira connector (Cloud and Data Center). The questionnaire (question sources 1–6). Confirmation card and audit log.

2. **Microsoft 365 and workflows across tools:** the Microsoft 365 connector (Teams, Outlook, To Do, OneDrive, directory). Plans that span Jira and Microsoft 365. Multi-step plans, e.g. an epic with its stories. Matching people across tools. Remembered preferences.
3. **Open ecosystem and smarter intake:** custom MCP connections, the ServiceNow connector, quality-question templates, and screenshot and clipboard capture.
4. **Deflection and automation:** answering from the knowledge base before a ticket is needed, duplicate detection, and proactive status notifications.

Market position

Our edge is three things together: a Windows hotkey assistant, support for Jira Data Center as well as Cloud, and checked questions plus confirmation before anything changes. The biggest threat is general AI desktop apps with vendor MCP servers becoming good enough.

Product	Overlap with us	Gap we exploit
Atlassian Rovo	Natural-language create and update in Jira	Atlassian-only, Cloud-only, no desktop hotkey, no enforced questionnaire
ChatGPT / Claude desktop + Rovo MCP	Nearly everything, for Jira Cloud	General-purpose, with no enforced clarification, confirmation or organisation audit
ServiceNow Now Assist + Moveworks	Employee "AI front door" and deflection	Enterprise pricing, ServiceNow-centric, tied to chat platforms
Aisera, Atomicwork	Conversational IT service across Jira and ServiceNow	Heavy platforms, not a lightweight personal assistant
PromptJ	Closest in form: hotkey-style chat to Jira actions	macOS-only, Jira-only, no clarification engine
Microsoft 365 Copilot Jira connector	Search over Jira	Read-only

This table comes from the market scan of 2026-10-07 and was not re-checked for this doc.

Tech stack options

Recommendation: Option A, TypeScript end to end on Electron. It is the fastest way to ship the MVP, and its ecosystem fits the core best: the MCP SDK, the Anthropic SDK, JSON Schema tooling and chat UI components. Choose Option C instead if the team is mainly C#/.NET, and Option B if install size and memory matter more than build speed.

	A. Electron + TypeScript	B. Tauri + TypeScript	C. .NET (WinUI 3) + C#	Python (PySide6)
Languages	TypeScript only	Rust shell, TypeScript UI and core	C# (optionally a web view for the chat)	Python
Install size (approx.)	Large, ~100 MB	Small, ~10 MB	Medium	Medium to large
Idle memory (approx.)	Highest of the four	Low	Low	Medium
Hotkey and tray	Built in	Official plugins	Win32 RegisterHotKey + tray library	Third-party libraries
Chat UI (cards, forms, live draft)	Easiest: React and web components	Easiest: same web UI	Hardest in XAML. Hosting the chat in WebView2 helps	Hard
Microsoft 365 sign-in	MSAL Node, with Windows broker support	MSAL Node in the sidecar	Best: MSAL.NET with Windows (WAM) single sign-on	MSAL Python
MCP client	Official TypeScript SDK, the reference implementation	Same, in the sidecar	Official C# SDK	Official Python SDK
Claude SDK	Official	Official (in the sidecar)	Official	Official
Core can move to a server	Yes, plain Node package	Yes, if the core stays in TypeScript	Yes, as ASP.NET	Yes
Packaging and updates	electron-builder (NSIS/MSI) + auto-updater	Built-in bundler + updater plugin	MSIX or Velopack	PyInstaller; updates are DIY

	A. Electron + TypeScript	B. Tauri + TypeScript	C. .NET (WinUI 3) + C#	Python (PySide6)
Main risk	Size and memory for an always-on tray app	Two runtimes (Rust + Node sidecar) to package and debug	Slower UI work; smaller AI sample ecosystem	Packaging, antivirus false positives, updates

Python is listed for completeness but not recommended for a shipped desktop app. Sizes and memory are rough, from general experience, and not measured for this app. Library facts outside the Claude row are from general knowledge as of mid-2026 and should be checked when the build starts.

Deciding factor still open: the team's main language. If the team writes C# daily, Option C's Microsoft 365 sign-in and Intune story outweigh A's faster UI work.

Tech stack by layer

The table shows each layer for Option A, with the alternatives that would replace it. One choice runs through every layer: **JSON Schema is the single schema language.** Jira fields are converted to it, MCP tools already use it, and Claude's structured outputs accept it. The questionnaire therefore validates every answer the same way, whatever the tool.

Layer	Option A pick	Alternatives	Notes
Chat UI	React in the Electron window	Svelte; WinUI 3 (Option C)	Question cards, confirmation cards and the draft panel are web components
Hotkey and tray	Electron globalShortcut + Tray	Tauri plugins; Win32 RegisterHotKey	Hotkey is configurable, with conflict detection
Core runtime	Node.js package in an Electron utility process	Rust (Tauri); .NET	Has no UI code. Talks to the shell through typed messages that could later go over HTTP or WebSocket

Layer	Option A pick	Alternatives	Notes
LLM	Claude via the Anthropic API	Claude via Microsoft Foundry, Amazon Bedrock or Google Vertex; Azure OpenAI; local models for strict no-cloud rules	Foundry lets Microsoft-centric companies buy Claude through Azure at standard API rates
Models	Claude Opus 5.5 (`claude-opus-5-5`, $4 / $20 per million input / output tokens)	Claude Haiku 5.5 ($0.10 / $0.50) for routing and classification; Claude Sonnet 5.5 ($2 / $10)	Measure Opus 5.5 at low effort against Haiku 5.5 for routing before splitting models
LLM gateway	Thin module of our own over the official SDK	Vercel AI SDK, if multi-provider is needed from day one	Uses structured outputs and strict tool schemas for intent and field extraction. No heavy agent framework: the state machine is ours
Jira client	Direct REST calls with typed wrappers, plus a Cloud / Data Center adapter	jira.js (community library); Atlassian Rovo MCP server for Cloud	No official Atlassian SDK. Cloud uses REST v3, Data Center REST v2
Jira sign-in	API token (Cloud), Personal Access Token (Data Center)	OAuth 2.0 (3LO) for Cloud	Open: whether a desktop app can use 3LO without shipping a client secret
Microsoft 365	MSAL Node (Windows broker) + Microsoft Graph SDK	MSAL.NET with WAM + Graph .NET SDK (Option C)	Needs an Entra ID app registration with delegated permissions. IT approves it once
MCP client	Official MCP TypeScript SDK	Official C# or Python SDK	Remote servers over HTTP, local servers over stdio

Layer	Option A pick	Alternatives	Notes
Schemas and validation	JSON Schema with Ajv, and Zod for internal types	NJsonSchema (.NET)	Shared by connectors, the questionnaire and LLM outputs
Credentials	Electron safeStorage (Windows DPAPI)	Windows Credential Manager; MSAL's encrypted token cache	Stored per connection. Never logged or sent to the LLM
Local data	SQLite (better-sqlite3) for audit log, schema cache and drafts. A JSON file for settings	Microsoft.Data.Sqlite	One file per user profile, easy to back up or wipe
Packaging	electron-builder: signed NSIS or MSI installer + electron-updater	MSIX; Velopack (.NET)	Needs a code-signing certificate. Intune deploys the MSI
Telemetry	Opt-in anonymous metrics through OpenTelemetry	Application Insights	Questions per task, confirm and cancel rates, failures
Testing	Vitest for the core, Playwright for the Electron UI, recorded Jira responses	xUnit (.NET)	Each connector is tested against recorded API fixtures

Claude model IDs and prices are from Anthropic's model table as of 2026-10-06. Everything else is general knowledge and should be checked when the build starts.

Next step: confirm the team's main language and the LLM data rules, then split the MVP into build tasks on the chosen stack.